Chapter 15. Metadata

Along with storing all of the data that various users insert into a database, a database server also needs to store information about all of the database objects (tables, views, indexes, etc.) that were created to store this data. The database server stores this information, not surprisingly, in a database. This chapter discusses how and where this information, known as *metadata*, is stored, how you can access it, and how you can use it to build flexible systems.

Data About Data

Metadata is essentially data about data. Every time you create a database object, the database server needs to record various pieces of information. For example, if you were to create a table with multiple columns, a primary key constraint, three indexes, and a foreign key constraint, the database server would need to store all the following information:

- Table name
- Table storage information (tablespace, initial size, etc.)
- Storage engine
- Column names
- Column data types
- Default column values
- `not null` column constraints
- Primary key columns
- Primary key name
- Name of primary key index
- Index names
- Index types (B-tree, bitmap)
- Indexed columns
- Index column sort order (ascending or descending)
- Index storage information
- Foreign key name
- Foreign key columns

- Associated table/columns for foreign keys

This data is collectively known as the *data dictionary* or *system catalog*. The database server needs to store this data persistently, and it needs to be able to quickly retrieve this data in order to verify and execute SQL statements. Additionally, the database server must safeguard this data so that it can be modified only via an appropriate mechanism, such as the `alter table` statement.

While standards exist for the exchange of metadata between different servers, every database server uses a different mechanism to publish metadata, such as:

- A set of views, such as Oracle Database's `user_tables` and `all_constraints` views
- A set of system-stored procedures, such as SQL Server's `sp_tables` procedure or Oracle Database's `dbms_metadata` package
- A special database, such as MySQL's `information_schema` database

Along with SQL Server's system-stored procedures, which are a vestige of its Sybase lineage, SQL Server also includes a special schema called `information_schema` that is provided automatically within each database. Both MySQL and SQL Server provide this interface to conform with the ANSI SQL:2003 standard. The remainder of this chapter discusses the `information_schema` objects that are available in MySQL and SQL Server.

information_schema

All of the objects available within the `information_schema` database (or schema, in the case of SQL Server) are views. Unlike the `describe` utility, which I used in several chapters of this book as a way to show the structure of various tables and views, the views within `information_schema` can be queried and, thus, used programmatically (more on this later in the chapter). Here's an example that demonstrates how to retrieve the names of all of the tables in the Sakila database:

```
mysql> SELECT table_name, table_type
    -> FROM information_schema.tables
    -> WHERE table_schema = 'sakila'
    -> ORDER BY 1;
+---------------------------+------------+
| TABLE_NAME                | TABLE_TYPE |
+---------------------------+------------+
| actor                     | BASE TABLE |
| actor_info                | VIEW       |
```

```
| address                | BASE TABLE |
| category               | BASE TABLE |
| city                   | BASE TABLE |
| country                | BASE TABLE |
| customer               | BASE TABLE |
| customer_list          | VIEW       |
| film                   | BASE TABLE |
| film_actor             | BASE TABLE |
| film_category          | BASE TABLE |
| film_list              | VIEW       |
| film_text              | BASE TABLE |
| inventory              | BASE TABLE |
| language               | BASE TABLE |
| nicer_but_slower_film_list | VIEW       |
| payment                | BASE TABLE |
| rental                 | BASE TABLE |
| sales_by_film_category | VIEW       |
| sales_by_store         | VIEW       |
| staff                  | BASE TABLE |
| staff_list             | VIEW       |
| store                  | BASE TABLE |
+----------------------------+------------+
23 rows in set (0.00 sec)
```

As you can see, the `information_schema.tables` view includes both tables and views; if you want to exclude the views, simply add another condition to the `where` clause:

```
mysql> SELECT table_name, table_type
    -> FROM information_schema.tables
    -> WHERE table_schema = 'sakila'
    ->   AND table_type = 'BASE TABLE'
    -> ORDER BY 1;
+---------------+------------+
| TABLE_NAME    | TABLE_TYPE |
+---------------+------------+
| actor         | BASE TABLE |
| address       | BASE TABLE |
| category      | BASE TABLE |
| city          | BASE TABLE |
| country       | BASE TABLE |
| customer      | BASE TABLE |
| film          | BASE TABLE |
| film_actor    | BASE TABLE |
| film_category | BASE TABLE |
| film_text     | BASE TABLE |
```

```
| inventory     | BASE TABLE |
| language      | BASE TABLE |
| payment       | BASE TABLE |
| rental        | BASE TABLE |
| staff         | BASE TABLE |
| store         | BASE TABLE |
+---------------+------------+
16 rows in set (0.00 sec)
```

If you are only interested in information about views, you can query `information_schema.views`. Along with the view names, you can retrieve additional information, such as a flag that shows whether a view is updatable:

```
mysql> SELECT table_name, is_updatable
    -> FROM information_schema.views
    -> WHERE table_schema = 'sakila'
    -> ORDER BY 1;
+----------------------------+--------------+
| TABLE_NAME                 | IS_UPDATABLE |
+----------------------------+--------------+
| actor_info                 | NO           |
| customer_list              | YES          |
| film_list                  | NO           |
| nicer_but_slower_film_list | NO           |
| sales_by_film_category     | NO           |
| sales_by_store             | NO           |
| staff_list                 | YES          |
+----------------------------+--------------+
7 rows in set (0.00 sec)
```

Column information for both tables and views is available via the `columns` view. The following query shows column information for the `film` table:

```
mysql> SELECT column_name, data_type,
    ->   character_maximum_length char_max_len,
    ->   numeric_precision num_prcsn, numeric_scale num_scale
    -> FROM information_schema.columns
    -> WHERE table_schema = 'sakila' AND table_name = 'film'
    -> ORDER BY ordinal_position;
+---------------------+-----------+-------------+-----------+-----------+
| COLUMN_NAME         | DATA_TYPE | char_max_len | num_prcsn | num_scale |
+---------------------+-----------+-------------+-----------+-----------+
| film_id             | smallint  |        NULL |         5 |         0 |
```

```
| title              | varchar   |        255 |     NULL |     NULL |
| description        | text      |      65535 |     NULL |     NULL |
| release_year       | year      |       NULL |     NULL |     NULL |
| language_id        | tinyint   |       NULL |        3 |        0 |
| original_language_id | tinyint   |       NULL |        3 |        0 |
| rental_duration    | tinyint   |       NULL |        3 |        0 |
| rental_rate        | decimal   |       NULL |        4 |        2 |
| length             | smallint  |       NULL |        5 |        0 |
| replacement_cost   | decimal   |       NULL |        5 |        2 |
| rating             | enum      |          5 |     NULL |     NULL |
| special_features   | set       |         54 |     NULL |     NULL |
| last_update        | timestamp |       NULL |     NULL |     NULL |
+--------------------+-----------+------------+----------+----------+
13 rows in set (0.00 sec)
```

The `ordinal_position` column is included merely as a means to retrieve the columns in the order in which they were added to the table.

You can retrieve information about a table's indexes via the `information_schema.statistics` view as demonstrated by the following query, which retrieves information for the indexes built on the `rental` table:

```
mysql> SELECT index_name, non_unique, seq_in_index, column_name
    -> FROM information_schema.statistics
    -> WHERE table_schema = 'sakila' AND table_name = 'rental'
    -> ORDER BY 1, 3;
+--------------------+------------+--------------+--------------+
| INDEX_NAME         | NON_UNIQUE | SEQ_IN_INDEX | COLUMN_NAME  |
+--------------------+------------+--------------+--------------+
| idx_fk_customer_id |          1 |            1 | customer_id  |
| idx_fk_inventory_id |          1 |            1 | inventory_id |
| idx_fk_staff_id    |          1 |            1 | staff_id     |
| PRIMARY            |          0 |            1 | rental_id    |
| rental_date        |          0 |            1 | rental_date  |
| rental_date        |          0 |            2 | inventory_id |
| rental_date        |          0 |            3 | customer_id  |
+--------------------+------------+--------------+--------------+
7 rows in set (0.02 sec)
```

The `rental` table has a total of five indexes, one of which has three columns (`rental_date`) and one of which is a unique index (`PRIMARY`) used for the primary key constraint.

You can retrieve the different types of constraints (foreign key, primary key, unique) that have been created via the `information_schema.table_constraints` view. Here's a query that retrieves all of the constraints in the Sakila schema:

```
mysql> SELECT constraint_name, table_name, constraint_type
    -> FROM information_schema.table_constraints
    -> WHERE table_schema = 'sakila'
    -> ORDER BY 3,1;
+--------------------------+--------------+----------------+
| constraint_name          | table_name   | constraint_type |
+--------------------------+--------------+----------------+
| fk_address_city          | address      | FOREIGN KEY    |
| fk_city_country          | city         | FOREIGN KEY    |
| fk_customer_address      | customer     | FOREIGN KEY    |
| fk_customer_store        | customer     | FOREIGN KEY    |
| fk_film_actor_actor      | film_actor   | FOREIGN KEY    |
| fk_film_actor_film       | film_actor   | FOREIGN KEY    |
| fk_film_category_category | film_category | FOREIGN KEY    |
| fk_film_category_film    | film_category | FOREIGN KEY    |
| fk_film_language         | film         | FOREIGN KEY    |
| fk_film_language_original | film         | FOREIGN KEY    |
| fk_inventory_film        | inventory    | FOREIGN KEY    |
| fk_inventory_store       | inventory    | FOREIGN KEY    |
| fk_payment_customer      | payment      | FOREIGN KEY    |
| fk_payment_rental        | payment      | FOREIGN KEY    |
| fk_payment_staff         | payment      | FOREIGN KEY    |
| fk_rental_customer       | rental       | FOREIGN KEY    |
| fk_rental_inventory      | rental       | FOREIGN KEY    |
| fk_rental_staff          | rental       | FOREIGN KEY    |
| fk_staff_address         | staff        | FOREIGN KEY    |
| fk_staff_store           | staff        | FOREIGN KEY    |
| fk_store_address         | store        | FOREIGN KEY    |
| fk_store_staff           | store        | FOREIGN KEY    |
| PRIMARY                  | film         | PRIMARY KEY    |
| PRIMARY                  | film_actor   | PRIMARY KEY    |
| PRIMARY                  | staff        | PRIMARY KEY    |
| PRIMARY                  | film_category | PRIMARY KEY    |
| PRIMARY                  | store        | PRIMARY KEY    |
| PRIMARY                  | actor        | PRIMARY KEY    |
| PRIMARY                  | film_text    | PRIMARY KEY    |
| PRIMARY                  | address      | PRIMARY KEY    |
| PRIMARY                  | inventory    | PRIMARY KEY    |
| PRIMARY                  | customer     | PRIMARY KEY    |
| PRIMARY                  | category     | PRIMARY KEY    |
| PRIMARY                  | language     | PRIMARY KEY    |
```

```
| PRIMARY                | city         | PRIMARY KEY    |
| PRIMARY                | payment      | PRIMARY KEY    |
| PRIMARY                | country      | PRIMARY KEY    |
| PRIMARY                | rental       | PRIMARY KEY    |
| idx_email              | customer     | UNIQUE         |
| idx_unique_manager     | store        | UNIQUE         |
| rental_date            | rental       | UNIQUE         |
+------------------------+--------------+----------------+
41 rows in set (0.02 sec)
```

Table 15-1 shows many of the `information_schema` views that are available in MySQL version 8.0.

Table 15-1. information_schema views

View name	Provides information about...
schemata	Databases
tables	Tables and views
columns	Columns of tables and views
statistics	Indexes
user_privileges	Who has privileges on which schema objects
schema_privileges	Who has privileges on which databases
table_privileges	Who has privileges on which tables
column_privileges	Who has privileges on which columns of which tables
character_sets	What character sets are available
collations	What collations are available for which character sets
collation_character_set_applicability	Which character sets are available for which collation
table_constraints	The unique, foreign key, and primary key constraints
key_column_usage	The constraints associated with each key column
routines	Stored routines (procedures and functions)
views	Views
triggers	Table triggers

View name	Provides information about...
`plugins`	Server plug-ins
engines	Available storage engines
`partitions`	Table partitions
events	Scheduled events
`processlist`	Running processes
referential_constraints	Foreign keys
`parameters`	Stored procedure and function parameters
profiling	User profiling information

While some of these views, such as `engines`, `events`, and `plugins`, are specific to MySQL, many of these views are available in SQL Server as well. If you are using Oracle Database, please consult the online Oracle Database Reference Guide for information about the `user_`, `all_`, and `dba_` views, as well as the `dbms_metadata` package.

Working with Metadata

As I mentioned earlier, having the ability to retrieve information about your schema objects via SQL queries opens up some interesting possibilities. This section shows several ways in which you can make use of metadata in your applications.

Schema Generation Scripts

While some project teams include a full-time database designer who oversees the design and implementation of the database, many projects take the "design-by-committee" approach, allowing multiple people to create database objects. After several weeks or months of development, you may need to generate a script that will create the various tables, indexes, views, and so on, that the team has deployed. Although a variety of tools and utilities will generate these types of scripts for you, you can also query the `information_schema` views and generate the script yourself.

As an example, let's build a script that will create the `sakila.category` table. Here's the command used to build the table, which I extracted from the script used to build the example database:

```
CREATE TABLE category (
  category_id TINYINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name VARCHAR(25) NOT NULL,
  last_update TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY  (category_id)
)ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

Although it would certainly be easier to generate the script with the use of a procedural language (e.g., Transact-SQL or Java), since this is a book about SQL, I'm going to write a single query that will generate the `create table` statement. The first step is to query the `information_schema.columns` table to retrieve information about the columns in the table:

```
mysql> SELECT 'CREATE TABLE category (' create_table_statement
    -> UNION ALL
    -> SELECT cols.txt
    -> FROM
    ->  (SELECT concat('  ',column_name, ' ', column_type,
    ->    CASE
    ->      WHEN is_nullable = 'NO' THEN ' not null'
    ->      ELSE ''
    ->    END,
    ->    CASE
    ->      WHEN extra IS NOT NULL AND extra LIKE 'DEFAULT_GENERATED%'
    ->       THEN concat(' DEFAULT ',column_default,substr(extra,18))
    ->      WHEN extra IS NOT NULL THEN concat(' ', extra)
    ->      ELSE ''
    ->    END,
    ->    ',') txt
    ->   FROM information_schema.columns
    ->   WHERE table_schema = 'sakila' AND table_name = 'category'
    ->   ORDER BY ordinal_position
    ->  ) cols
    -> UNION ALL
    -> SELECT ')';
+-----------------------------------------------------------------------+
| create_table_statement                                                |
+-----------------------------------------------------------------------+
```

```
| CREATE TABLE category (                                          |
|   category_id tinyint(3) unsigned not null auto_increment,       |
|   name varchar(25) not null ,                                    |
|   last_update timestamp not null DEFAULT CURRENT_TIMESTAMP       |
|    on update CURRENT_TIMESTAMP,                                  |
| )                                                                |
+------------------------------------------------------------------+
5 rows in set (0.00 sec)
```

Well, that got us pretty close; we just need to add queries against the `table_constraints` and `key_column_usage` views to retrieve information about the primary key constraint:

```
mysql> SELECT 'CREATE TABLE category (' create_table_statement
    -> UNION ALL
    -> SELECT cols.txt
    -> FROM
    ->  (SELECT concat('  ',column_name, ' ', column_type,
    ->    CASE
    ->      WHEN is_nullable = 'NO' THEN ' not null'
    ->      ELSE ''
    ->    END,
    ->    CASE
    ->      WHEN extra IS NOT NULL AND extra LIKE 'DEFAULT_GENERATED%'
    ->        THEN concat(' DEFAULT ',column_default,substr(extra,18))
    ->      WHEN extra IS NOT NULL THEN concat(' ', extra)
    ->      ELSE ''
    ->    END,
    ->    ',') txt
    ->   FROM information_schema.columns
    ->   WHERE table_schema = 'sakila' AND table_name = 'category'
    ->   ORDER BY ordinal_position
    ->  ) cols
    -> UNION ALL
    -> SELECT concat('  constraint primary key (')
    -> FROM information_schema.table_constraints
    -> WHERE table_schema = 'sakila' AND table_name = 'category'
    ->   AND constraint_type = 'PRIMARY KEY'
    -> UNION ALL
    -> SELECT cols.txt
    -> FROM
    ->  (SELECT concat(CASE WHEN ordinal_position > 1 THEN '   ,'
    ->     ELSE '    ' END, column_name) txt
    ->   FROM information_schema.key_column_usage
    ->   WHERE table_schema = 'sakila' AND table_name = 'category'
    ->     AND constraint_name = 'PRIMARY'
```

```
    ->   ORDER BY ordinal_position
    ->  ) cols
    -> UNION ALL
    -> SELECT '  )'
    -> UNION ALL
    -> SELECT ')';
+-----------------------------------------------------------------------+
| create_table_statement                                                |
+-----------------------------------------------------------------------+
| CREATE TABLE category (                                               |
|   category_id tinyint(3) unsigned not null auto_increment,            |
|   name varchar(25) not null ,                                         |
|   last_update timestamp not null DEFAULT CURRENT_TIMESTAMP            |
|     on update CURRENT_TIMESTAMP,                                      |
|   constraint primary key (                                            |
|     category_id                                                       |
|   )                                                                   |
| )                                                                     |
+-----------------------------------------------------------------------+
8 rows in set (0.02 sec)
```

To see whether the statement is properly formed, I'll paste the query output into the `mysql` tool (I've changed the table name to `category2` so that it won't step on our existing table):

```
mysql> CREATE TABLE category2 (
    ->   category_id tinyint(3) unsigned not null auto_increment,
    ->   name varchar(25) not null ,
    ->   last_update timestamp not null DEFAULT CURRENT_TIMESTAMP
    ->     on update CURRENT_TIMESTAMP,
    ->   constraint primary key (
    ->     category_id
    ->   )
    -> );
Query OK, 0 rows affected (0.61 sec)
```

The statement executed without errors, and there is now a `category2` table in the Sakila database. For the query to generate a well-formed `create table` statement for *any* table, more work is required (such as handling indexes and foreign key constraints), but I'll leave that as an exercise.

NOTE

If you are using a graphical development tool such as Toad, Oracle SQL Developer, or MySQL Workbench, you will be able to easily generate these types of scripts without writing your own queries. But, just in case you are stuck on a deserted island with only the MySQL command-line client...

Deployment Verification

Many organizations allow for database maintenance windows, wherein existing database objects may be administered (such as adding/dropping partitions) and new schema objects and code can be deployed. After the deployment scripts have been run, it's a good idea to run a verification script to ensure that the new schema objects are in place with the appropriate columns, indexes, primary keys, and so forth. Here's a query that returns the number of columns, number of indexes, and number of primary key constraints (`0` or `1`) for each table in the Sakila schema:

```
mysql> SELECT tbl.table_name,
    ->  (SELECT count(*) FROM information_schema.columns clm
    ->   WHERE clm.table_schema = tbl.table_schema
    ->     AND clm.table_name = tbl.table_name) num_columns,
    ->  (SELECT count(*) FROM information_schema.statistics sta
    ->   WHERE sta.table_schema = tbl.table_schema
    ->     AND sta.table_name = tbl.table_name) num_indexes,
    ->  (SELECT count(*) FROM information_schema.table_constraints tc
    ->   WHERE tc.table_schema = tbl.table_schema
    ->     AND tc.table_name = tbl.table_name
    ->     AND tc.constraint_type = 'PRIMARY KEY') num_primary_keys
    -> FROM information_schema.tables tbl
    -> WHERE tbl.table_schema = 'sakila' AND tbl.table_type = 'BASE TABLE'
    -> ORDER BY 1;
+--------------+-------------+-------------+-----------------+
| TABLE_NAME   | num_columns | num_indexes | num_primary_keys |
+--------------+-------------+-------------+-----------------+
| actor        |           4 |           2 |               1 |
| address      |           9 |           3 |               1 |
| category     |           3 |           1 |               1 |
| city         |           4 |           2 |               1 |
| country      |           3 |           1 |               1 |
| customer     |           9 |           7 |               1 |
| film         |          13 |           4 |               1 |
| film_actor   |           3 |           3 |               1 |
| film_category |          3 |           3 |               1 |
```

```
| film_text     |         3 |         3 |             1 |
| inventory     |         4 |         4 |             1 |
| language      |         3 |         1 |             1 |
| payment       |         7 |         4 |             1 |
| rental        |         7 |         7 |             1 |
| staff         |        11 |         3 |             1 |
| store         |         4 |         3 |             1 |
+---------------+-----------+-----------+-----------------+
16 rows in set (0.01 sec)
```

You could execute this statement before and after the deployment and then verify any differences between the two sets of results before declaring the deployment a success.

Dynamic SQL Generation

Some languages, such as Oracle's PL/SQL and Microsoft's Transact-SQL, are supersets of the SQL language, meaning that they include SQL statements in their grammar along with the usual procedural constructs, such as "if-then-else" and "while." Other languages, such as Java, include the ability to interface with a relational database but do not include SQL statements in the grammar, meaning that all SQL statements must be contained within strings.

Therefore, most relational database servers, including SQL Server, Oracle Database, and MySQL, allow SQL statements to be submitted to the server as strings. Submitting strings to a database engine rather than utilizing its SQL interface is generally known as *dynamic SQL execution*. Oracle's PL/SQL language, for example, includes an `execute immediate` command, which you can use to submit a string for execution, while SQL Server includes a system stored procedure called `sp_executesql` for executing SQL statements dynamically.

MySQL provides the statements `prepare`, `execute`, and `deallocate` to allow for dynamic SQL execution. Here's a simple example:

```
mysql> SET @qry = 'SELECT customer_id, first_name, last_name FROM customer';
Query OK, 0 rows affected (0.00 sec)

mysql> PREPARE dynsql1 FROM @qry;
Query OK, 0 rows affected (0.00 sec)
Statement prepared

mysql> EXECUTE dynsql1;
```

```
+-------------+------------+-------------+
| customer_id | first_name  | last_name    |
+-------------+------------+-------------+
|         505 | RAFAEL     | ABNEY       |
|         504 | NATHANIEL   | ADAM        |
|          36 | KATHLEEN    | ADAMS       |
|          96 | DIANA      | ALEXANDER    |
...
|          31 | BRENDA     | WRIGHT      |
|         318 | BRIAN      | WYMAN       |
|         402 | LUIS       | YANEZ       |
|         413 | MARVIN     | YEE         |
|          28 | CYNTHIA    | YOUNG       |
+-------------+------------+-------------+
599 rows in set (0.02 sec)

mysql> DEALLOCATE PREPARE dynsql1;
Query OK, 0 rows affected (0.00 sec)
```

The `set` statement simply assigns a string to the `qry` variable, which is then submitted to the database engine (for parsing, security checking, and optimization) using the `prepare` statement. After executing the statement by calling `execute`, the statement must be closed using `deallocate prepare`, which frees any database resources (e.g., cursors) that have been utilized during execution.

The next example shows how you could execute a query that includes placeholders so that conditions can be specified at runtime:

```
mysql> SET @qry = 'SELECT customer_id, first_name, last_name
  FROM customer WHERE customer_id = ?';
Query OK, 0 rows affected (0.00 sec)

mysql> PREPARE dynsql2 FROM @qry;
Query OK, 0 rows affected (0.00 sec)
Statement prepared

mysql> SET @custid = 9;
Query OK, 0 rows affected (0.00 sec)

mysql> EXECUTE dynsql2 USING @custid;
+-------------+------------+-----------+
| customer_id | first_name | last_name |
+-------------+------------+-----------+
|           9 | MARGARET   | MOORE     |
```

```
+-------------+------------+-----------+
1 row in set (0.00 sec)

mysql> SET @custid = 145;
Query OK, 0 rows affected (0.00 sec)

mysql> EXECUTE dynsql2 USING @custid;
+-------------+------------+-----------+
| customer_id | first_name | last_name |
+-------------+------------+-----------+
|         145 | LUCILLE    | HOLMES    |
+-------------+------------+-----------+
1 row in set (0.00 sec)

mysql> DEALLOCATE PREPARE dynsql2;
Query OK, 0 rows affected (0.00 sec)
```

In this sequence, the query contains a placeholder (the `?` at the end of the statement) so that the customer ID value can be submitted at runtime. The statement is prepared once and then executed twice, once for customer ID 9 and again for customer ID 145, after which the statement is closed.

What, you may wonder, does this have to do with metadata? Well, if you are going to use dynamic SQL to query a table, why not build the query string using metadata rather than hardcoding the table definition? The following example generates the same dynamic SQL string as the previous example, but it retrieves the column names from the `information_schema.columns` view:

```
mysql> SELECT concat('SELECT ',
    ->   concat_ws(',', cols.col1, cols.col2, cols.col3, cols.col4,
    ->     cols.col5, cols.col6, cols.col7, cols.col8, cols.col9),
    ->   ' FROM customer WHERE customer_id = ?')
    -> INTO @qry
    -> FROM
    ->  (SELECT
    ->     max(CASE WHEN ordinal_position = 1 THEN column_name
    ->       ELSE NULL END) col1,
    ->     max(CASE WHEN ordinal_position = 2 THEN column_name
    ->       ELSE NULL END) col2,
    ->     max(CASE WHEN ordinal_position = 3 THEN column_name
    ->       ELSE NULL END) col3,
    ->     max(CASE WHEN ordinal_position = 4 THEN column_name
    ->       ELSE NULL END) col4,
```

```
    ->     max(CASE WHEN ordinal_position = 5 THEN column_name
    ->       ELSE NULL END) col5,
    ->     max(CASE WHEN ordinal_position = 6 THEN column_name
    ->       ELSE NULL END) col6,
    ->     max(CASE WHEN ordinal_position = 7 THEN column_name
    ->       ELSE NULL END) col7,
    ->     max(CASE WHEN ordinal_position = 8 THEN column_name
    ->       ELSE NULL END) col8,
    ->     max(CASE WHEN ordinal_position = 9 THEN column_name
    ->       ELSE NULL END) col9
    ->   FROM information_schema.columns
    ->   WHERE table_schema = 'sakila' AND table_name = 'customer'
    ->   GROUP BY table_name
    ->  ) cols;
Query OK, 1 row affected (0.00 sec)
mysql> SELECT @qry;
+-------------------------------------------------------------------+
| @qry                                                              |
+-------------------------------------------------------------------+
| SELECT customer_id,store_id,first_name,last_name,email,
    address_id,active,create_date,last_update
  FROM customer WHERE customer_id = ?                               |
+-------------------------------------------------------------------+
1 row in set (0.00 sec)

mysql> PREPARE dynsql3 FROM @qry;
Query OK, 0 rows affected (0.00 sec)
Statement prepared

mysql> SET @custid = 45;
Query OK, 0 rows affected (0.00 sec)

mysql> EXECUTE dynsql3 USING @custid;
+-------------+----------+------------+-----------+
| customer_id | store_id | first_name | last_name
+-------------+----------+------------+-----------+
|          45 |        1 | JANET      | PHILLIPS
+-------------+----------+------------+-----------+

  +---------------------------------+------------+--------
  | email                           | address_id | active
  +---------------------------------+------------+--------
  | JANET.PHILLIPS@sakilacustomer.org | 49       |      1
  +---------------------------------+------------+--------

  +--------------------+--------------------+
```

```
| create_date         | last_update         |
+---------------------+---------------------+
| 2006-02-14 22:04:36 | 2006-02-15 04:57:20 |
+---------------------+---------------------+
1 row in set (0.00 sec)

mysql> DEALLOCATE PREPARE dynsql3;
Query OK, 0 rows affected (0.00 sec)
```

The query pivots the first nine columns in the `customer` table, builds a query string using the `concat` and `concat_ws` functions, and assigns the string to the `qry` variable. The query string is then executed as before.

NOTE

Generally, it would be better to generate the query using a procedural language that includes looping constructs, such as Java, PL/SQL, Transact-SQL, or MySQL's Stored Procedure Language. However, I wanted to demonstrate a pure SQL example, so I had to limit the number of columns retrieved to some reasonable number, which in this example is nine.

Test Your Knowledge

The following exercises are designed to test your understanding of metadata. When you're finished, see Appendix B for the solutions.

Exercise 15-1

Write a query that lists all of the indexes in the Sakila schema. Include the table names.

Exercise 15-2

Write a query that generates output that can be used to create all of the indexes on the `sakila.customer` table. Output should be of the form:

```
"ALTER TABLE <table_name> ADD INDEX <index_name> (<column_list>)"
```

Previous chapter

< 14. Views

Next chapter

16. Analytic Functions >